SwiftUI Buttons: From Zero to Wow

A Hands-On Teaching Guide for Absolute Beginners

How to Use This Guide

This guide walks through building SwiftUI buttons **one concept at a time**. Each section introduces ONE new idea, with a working code example students can type into Xcode and see immediately.

The magic trick: **students see something cool on screen within 2 minutes of starting.**

Every section follows the same pattern:

1. Here's what we're building (show the result first)
2. Here's the code
3. Here's what each piece does (line-by-line)
4. Try changing this... (a mini challenge)

> ***Xcode Setup:*** *File > New > Project > iOS > App > SwiftUI. All code goes in* `ContentView.swift` .

Part 1: Your First Button

The Concept

A SwiftUI `Button` has two parts: **what it does** (the action) and **what it looks like** (the label).

The Code

```
import SwiftUI

struct ContentView: View {
    var body: some View {
        Button(action: {
            print("Button tapped!")
        }) {
            Text("Tap Me")
        }
    }
}
```

Line by Line

Line	What it does
`Button(action: { ... })`	Creates a button. The code inside `{ }` runs when tapped.
`print("Button tapped!")`	Prints a message to the Xcode console (bottom panel).
`Text("Tap Me")`	The visible part of the button -- what the user sees.

Try This

- Change `"Tap Me"` to your name.

- Change the `print` message. Tap the button and check the console output.

> ***Key takeaway:*** *A button = an action + a label. That's it. Everything else is decoration.*

Part 2: Making It Look Like a Real Button

The Concept

Plain text doesn't feel like a button. We make it *look* like one using **padding** (space inside), a **background** (the colored shape behind it), and **corner radius** (rounded corners).

The Code

```
Button(action: {
    print("Tapped!")
}) {
    Text("PRESS ME!")
        .font(.system(size: 20, weight: .bold, design: .rounded))
        .foregroundColor(.white)
        .padding(.horizontal, 36)
        .padding(.vertical, 16)
        .background(
            RoundedRectangle(cornerRadius: 14)
                .fill(Color.green)
        )
}
```

What's New Here

Modifier	What it does
`.font(...)`	Sets the text size (20), weight (bold), and style (rounded).
`.foregroundColor(.white)`	Makes the text white.
`.padding(.horizontal, 36)`	Adds 36 points of space on the left and right.
`.padding(.vertical, 16)`	Adds 16 points of space on top and bottom.
`.background(...)`	Places a shape behind the text.
`RoundedRectangle(cornerRadius: 14)`	A rectangle with rounded corners.
`.fill(Color.green)`	Fills the shape with green.

Think of It Like This

Imagine wrapping a gift:

- The **Text** is the gift
- **Padding** is the bubble wrap
- **Background** is the wrapping paper
- **Corner radius** rounds the edges of the box

Try This

- Change `Color.green` to `Color.blue`, `Color.orange`, or `Color.pink`.
- Change the `cornerRadius` to `0` (sharp corners) or `30` (very round).
- What happens if you set padding to `0`?

Part 3: Adding Depth with Shadows

The Concept

Real buttons feel like they sit *above* the surface. A **shadow** creates that illusion.

The Code

```
Button(action: {
    print("Tapped!")
}) {
    Text("PRESS ME!")
        .font(.system(size: 20, weight: .bold, design: .rounded))
        .foregroundColor(.white)
        .padding(.horizontal, 36)
        .padding(.vertical, 16)
        .background(
            RoundedRectangle(cornerRadius: 14)
                .fill(Color.green)
        )
        .shadow(color: Color.green.opacity(0.5), radius: 10, y: 5)
}
```

What's New

Parameter	What it does
`color:`	The shadow color. Using the same color as the button + `opacity` gives a colored glow.
`radius:`	How blurry/spread out the shadow is.
`y:`	Pushes the shadow downward, like a light shining from above.

Try This

- Remove `y: 5` -- what happens? (The shadow spreads evenly.)
- Change `radius` to `30` -- it becomes a glow.
- Try `Color.black.opacity(0.3)` for a subtle, realistic shadow.

Part 4: The Fake 3D Button (Duolingo Style)

The Concept

This is the trick Duolingo uses. You stack two rectangles: a **darker one behind** (the side/shadow) and a **brighter one in front** (the face). When pressed, the front slides down to meet the back.

The Code

From the app: `ThreeDPushButton.swift`

```
struct ThreeDPushButton: View {
    @State private var isPressed = false

    let frontColor = Color(red: 0.3, green: 0.85, blue: 0.4)
    let sideColor = Color(red: 0.15, green: 0.55, blue: 0.2)

    var body: some View {
        Button(action: {
            withAnimation(.spring(response: 0.15, dampingFraction: 0.5)) {
                isPressed = true
            }
            DispatchQueue.main.asyncAfter(deadline: .now() + 0.15) {
                withAnimation(.spring(response: 0.3, dampingFraction: 0.5)) {
                    isPressed = false
                }
            }
        }) {
            Text("PRESS ME!")
                .font(.system(size: 20, weight: .heavy, design: .rounded))
                .foregroundColor(.white)
                .padding(.horizontal, 36)
                .padding(.vertical, 16)
                .background(
                    ZStack {
                        // The "side" -- darker, sits behind
                        RoundedRectangle(cornerRadius: 14)
                            .fill(sideColor)
                            .offset(y: 6)

                        // The "front face" -- moves down when pressed
                        RoundedRectangle(cornerRadius: 14)
                            .fill(frontColor)
                            .offset(y: isPressed ? 4 : 0)
                    }
                )
                .offset(y: isPressed ? 4 : 0)
        }
    }
}
```

Three New Big Ideas

1. `@State` -- Remembering things

```
@State private var isPressed = false
```

This creates a variable that SwiftUI *watches*. When it changes, the screen updates automatically. Think of it as a light switch -- when you flip it, the room changes.

2. `withAnimation` -- Making changes smooth

```
withAnimation(.spring(response: 0.15, dampingFraction: 0.5)) {
    isPressed = true
```

```
}
```

Without this, the button would *teleport* between positions. `withAnimation` makes it *slide* there instead. The `.spring` makes it bouncy.

3. `DispatchQueue.main.asyncAfter` -- Doing something later

```
DispatchQueue.main.asyncAfter(deadline: .now() + 0.15) {
    isPressed = false
}
```

This says: "Wait 0.15 seconds, then set `isPressed` back to false." It's how we auto-release the button.

How the 3D Trick Works

```
Normal:                    Pressed:
 ____________                ____________
|            |  <- front    |            |
|  PRESS ME  |              |  PRESS ME  |  <- front moved down
|____________|              |____________|
|////////////|  <- side     |////////////|  <- side (stays put)
```

The `ZStack` layers the side behind the front. When pressed, the front's `offset` changes from `0` to `4`, sliding it down toward the side.

Try This

- Change `offset(y: 6)` on the side to `12` -- taller button.
- Change the spring `dampingFraction` to `0.2` (bouncier) or `0.9` (stiffer).
- Swap `frontColor` and `sideColor` -- looks weird, right? The darker color must be behind.

Part 5: Icons + Text Together

The Concept

Most real buttons have an **icon and text side by side**. In SwiftUI, `HStack` places things horizontally. SF Symbols gives you thousands of free icons.

The Code

```
Button(action: {
    print("Tapped!")
}) {
    HStack(spacing: 12) {
        Image(systemName: "power")
            .font(.system(size: 20, weight: .semibold))
        Text("Power On")
            .font(.system(size: 18, weight: .semibold, design: .rounded))
    }
    .foregroundColor(.white)
    .padding(.horizontal, 36)
    .padding(.vertical, 18)
    .background(
```

```
        RoundedRectangle(cornerRadius: 16)
            .fill(Color(red: 0.22, green: 0.24, blue: 0.29))
    )
    .shadow(color: .black.opacity(0.4), radius: 8, x: 4, y: 4)
}
```

What's New

Code	What it does
`HStack(spacing: 12)`	Lays out children side by side, with 12 points between them.
`Image(systemName: "power")`	Shows an SF Symbol icon.

> ***Finding icons:*** *In Xcode, go to the menu bar: Editor > Show SF Symbols Library. There are 5,000+ icons to choose from.*

Try This

- Replace `"power"` with `"heart.fill"`, `"star.fill"`, or `"bolt.fill"`.
- Change `HStack` to `VStack` -- the icon goes above the text.
- Add a second `Image` after `Text` to get an icon on both sides.

Part 6: Gradients (Multi-Color Backgrounds)

The Concept

Instead of a flat color, a **gradient** blends between two or more colors. It instantly makes a button look more polished.

The Code

```
Button(action: {
    print("Tapped!")
}) {
    Text("Flow")
        .font(.system(size: 22, weight: .bold, design: .rounded))
        .foregroundColor(.white)
        .padding(.horizontal, 48)
        .padding(.vertical, 18)
        .background(
            RoundedRectangle(cornerRadius: 16)
                .fill(
                    LinearGradient(
                        colors: [.orange, .pink, .purple],
                        startPoint: .leading,
                        endPoint: .trailing
                    )
                )
        )
        .shadow(color: .pink.opacity(0.5), radius: 15, y: 5)
}
```

What's New

Code	What it does
`LinearGradient(...)`	Blends colors in a straight line.
`colors: [.orange, .pink, .purple]`	The colors to blend through, in order.
`startPoint: .leading`	Start the gradient on the left.
`endPoint: .trailing`	End it on the right.

Other Directions You Can Use

Start	End	Result
`.leading`	`.trailing`	Left to right
`.top`	`.bottom`	Top to bottom
`.topLeading`	`.bottomTrailing`	Diagonal

Try This

- Use just 2 colors: `[.blue, .purple]`.
- Change direction to `.top` / `.bottom`.
- Try 5 colors -- rainbow button!
- Replace `LinearGradient` with `RadialGradient(colors: [...], center: .center, startRadius: 0, endRadius: 80)` for a circular gradient.

Part 7: Borders and Overlays

The Concept

An **overlay** places something *on top of* a view. It's commonly used to add a border stroke that sits over the background.

The Code

```
Button(action: {
    print("Tapped!")
}) {
    Text("Glass Effect")
        .font(.system(size: 18, weight: .medium, design: .rounded))
        .foregroundColor(.white)
        .padding(.horizontal, 32)
        .padding(.vertical, 18)
        .background(
            RoundedRectangle(cornerRadius: 20)
                .fill(Color.blue.opacity(0.3))
        )
        .overlay(
            RoundedRectangle(cornerRadius: 20)
                .stroke(
                    LinearGradient(
                        colors: [.white.opacity(0.4), .white.opacity(0.1)],
                        startPoint: .topLeading,
                        endPoint: .bottomTrailing
```

```
                    ),
                    lineWidth: 1
                )
        )
}
```

What's New

Code	What it does
`.overlay(...)`	Places a view on top. Here it adds a glowing border.
`.stroke(...)`	Draws just the outline of the shape (not filled).
`lineWidth: 1`	How thick the border is.
`.opacity(0.3)`	Makes the color 30% visible (semi-transparent).

`.overlay` vs `.background`

- `.background(...)` -- puts something **behind** the view
- `.overlay(...)` -- puts something **on top of** the view

Both match the size of the view they're attached to.

Try This

- Change `lineWidth` to `3` -- thicker border.
- Make the background darker and the border brighter for a neon look.
- Stack TWO overlays -- one stroke, one with a `fill` at very low opacity.

Part 8: Press Animation with `@State`

The Concept

Now we combine everything. The button **reacts to touch** -- it shrinks when pressed and bounces back. This is the core pattern used by every button in the Buttons App.

The Code

From the app: `NeumorphicButton.swift` *(simplified)*

```
struct MyButton: View {
    @State private var isPressed = false

    var body: some View {
        Button(action: {
            // Step 1: Press down
            withAnimation(.spring(response: 0.2, dampingFraction: 0.6)) {
                isPressed = true
            }
            // Step 2: Release after a short delay
            DispatchQueue.main.asyncAfter(deadline: .now() + 0.2) {
                withAnimation(.spring(response: 0.4, dampingFraction: 0.6)) {
                    isPressed = false
                }
```

```
            }
        }) {
            HStack(spacing: 12) {
                Image(systemName: "power")
                    .font(.system(size: 20, weight: .semibold))
                Text("Power On")
                    .font(.system(size: 18, weight: .semibold, design: .rounded))
            }
            .foregroundColor(.white.opacity(isPressed ? 0.5 : 0.8))
            .padding(.horizontal, 36)
            .padding(.vertical, 18)
            .background(
                RoundedRectangle(cornerRadius: 16)
                    .fill(Color(red: 0.22, green: 0.24, blue: 0.29))
                    .shadow(color: .black.opacity(isPressed ? 0 : 0.5),
                            radius: isPressed ? 2 : 8,
                            x: isPressed ? 0 : 6,
                            y: isPressed ? 0 : 6)
            )
            .scaleEffect(isPressed ? 0.97 : 1.0)
        }
    }
}
```

The Pattern (Use This For Every Button You Build)

```
1. Create @State var isPressed = false
2. On tap:  set isPressed = true  (with animation)
3. After delay:  set isPressed = false  (with animation)
4. In the label:  use isPressed to change appearance
```

What Changes When `isPressed` Is True?

Property	Normal	Pressed
Text opacity	0.8	0.5 (dimmer)
Shadow radius	8	2 (tighter)
Shadow offset	6,6	0,0 (closer)
Scale	1.0	0.97 (slightly smaller)

These small changes together create the illusion of physically pushing the button into the screen.

Try This

- Change `0.97` to `0.8` -- way too much squish. Find the sweet spot.
- Try adding `.rotation3DEffect(.degrees(isPressed ? 5 : 0), axis: (x: 1, y: 0, z: 0))` for a tilt on press.
- Remove the shadow changes but keep the scale -- still feels good!

Part 9: Continuous Animation (Making Things Move on Their Own)

The Concept

So far, animation only happens when you tap. But what if the button should **pulse**, **glow**, or **shimmer** all the time? Use `.onAppear` with `.repeatForever`.

The Code

```
struct PulsingButton: View {
    @State private var isPulsing = false

    var body: some View {
        Button(action: {
            print("Tapped!")
        }) {
            Text("ACTIVATE")
                .font(.system(size: 20, weight: .bold, design: .monospaced))
                .foregroundColor(Color(red: 0, green: 1, blue: 0.8))
                .padding(.horizontal, 40)
                .padding(.vertical, 20)
                .background(
                    RoundedRectangle(cornerRadius: 4)
                        .fill(Color.black.opacity(0.8))
                )
                .overlay(
                    RoundedRectangle(cornerRadius: 4)
                        .stroke(Color(red: 0, green: 1, blue: 0.8), lineWidth: 2)
                )
                .shadow(
                    color: Color(red: 0, green: 1, blue: 0.8).opacity(0.6),
                    radius: isPulsing ? 30 : 10
                )
        }
        .onAppear {
            withAnimation(
                .easeInOut(duration: 1.5)
                .repeatForever(autoreverses: true)
            ) {
                isPulsing = true
            }
        }
    }
}
```

What's New

Code	What it does
`.onAppear { }`	Runs code the moment this view appears on screen.
`.repeatForever(autoreverses: true)`	The animation plays forward, then backward, forever.
`radius: isPulsing ? 30 : 10`	The glow radius bounces between 10 and 30.

How `.onAppear` + `.repeatForever` Works

```
View appears
    |
    v
isPulsing = false ——animation——> isPulsing = true
                                        |
                            autoreverses back to false
                                        |
                                and repeats forever...
```

The shadow radius smoothly animates between 10 and 30, creating a breathing glow.

Try This

- Change `duration: 1.5` to `0.5` (fast heartbeat) or `3.0` (slow breathing).
- Instead of shadow, try animating `.scaleEffect(isPulsing ? 1.05 : 1.0)` -- a subtle pulse.
- Set `autoreverses: false` -- the animation snaps back instead of easing.

Part 10: Put It All Together -- Build Your Own Button

The Challenge

Using everything you've learned, build a button that has:

- [ ] An icon and text (`HStack` + `Image(systemName:)`)
- [ ] A gradient background (`LinearGradient`)
- [ ] Rounded corners (`RoundedRectangle(cornerRadius:)`)
- [ ] A shadow
- [ ] A press animation (`@State` , `withAnimation` , `scaleEffect`)
- [ ] A continuous glow or pulse (`.onAppear` + `.repeatForever`)

Starter Template

```
struct MyCustomButton: View {
    @State private var isPressed = false
    @State private var isGlowing = false

    var body: some View {
        Button(action: {
            // Press animation
            withAnimation(.spring(response: 0.2, dampingFraction: 0.6)) {
                isPressed = true
            }
            DispatchQueue.main.asyncAfter(deadline: .now() + 0.2) {
                withAnimation(.spring(response: 0.4, dampingFraction: 0.6)) {
                    isPressed = false
                }
            }
        }) {
            HStack(spacing: 10) {
                Image(systemName: "star.fill")
```

```
                    .font(.system(size: 18, weight: .bold))
                Text("My Button")
                    .font(.system(size: 20, weight: .bold, design: .rounded))
            }
            .foregroundColor(.white)
            .padding(.horizontal, 36)
            .padding(.vertical, 18)
            .background(
                RoundedRectangle(cornerRadius: 16)
                    .fill(
                        LinearGradient(
                            colors: [/* YOUR COLORS */],
                            startPoint: .leading,
                            endPoint: .trailing
                        )
                    )
            )
            .overlay(
                RoundedRectangle(cornerRadius: 16)
                    .stroke(.white.opacity(0.2), lineWidth: 1)
            )
            .shadow(
                color: /* YOUR SHADOW COLOR */.opacity(0.5),
                radius: isGlowing ? 25 : 12,
                y: 5
            )
            .scaleEffect(isPressed ? 0.94 : 1.0)
        }
        .onAppear {
            withAnimation(.easeInOut(duration: 1.5).repeatForever(autoreverses: true)) {
                isGlowing = true
            }
        }
    }
}
```

Fill in the `/* YOUR COLORS */` and `/* YOUR SHADOW COLOR */` and you've built a fully animated, professional-looking button.

Quick Reference Card

Modifiers Cheat Sheet

Want to...	Use this
Change text size/weight	`.font(.system(size: 20, weight: .bold))`
Change text color	`.foregroundColor(.white)`
Add space inside	`.padding(.horizontal, 36)`
Add a background shape	`.background(RoundedRectangle(...).fill(...))`
Add a border	`.overlay(RoundedRectangle(...).stroke(...))`

Add a shadow	`.shadow(color:radius:x:y:)`
Scale on press	`.scaleEffect(isPressed ? 0.95 : 1.0)`
Round the corners	`RoundedRectangle(cornerRadius: 16)`
Make semi-transparent	`.opacity(0.5)`

The Universal Button Pattern

```
@State private var isPressed = false

Button(action: {
    withAnimation(.spring(response: 0.2, dampingFraction: 0.6)) { isPressed = true }
    DispatchQueue.main.asyncAfter(deadline: .now() + 0.2) {
        withAnimation(.spring(response: 0.4)) { isPressed = false }
    }
}) {
    Text("Label")
        // ... styling ...
        .scaleEffect(isPressed ? 0.95 : 1.0)
}
```

What to Explore Next

Once comfortable with these basics, look at the more advanced buttons in the app:

Button	New Concept
`ElasticJellyButton`	Multi-step chained animation (squish + wobble + settle)
`RippleEffectButton`	Showing/hiding views conditionally
`ParticleBurstButton`	Dynamic arrays of views with `ForEach`
`GlassmorphicButton`	Layering materials + shimmer sweep
`NeonGlowButton`	Timer-driven animation with Combine
`PulseRingButton`	Spawning and removing animated elements

Built using the Buttons App -- 20 handcrafted SwiftUI buttons.